

DKail Security Monitor

开发与安装手册

版本：1.0.0

技术栈：Java 17 + Spring Boot 3.2 + React 18 + Tauri 2

开发者：陈美龙

小组成员：范心予，弓锦添，许堉红，漆清秀

文档日期：2026-06-12

AI Agent Collaborative Development

目录

第一部分：项目概述

1.1 项目定位

1.2 功能模块

1.3 技术架构

第二部分：AI Agent 协同开发过程

Step 1 — 需求分析与 PRD 编写

Step 2 — 伪代码设计与技术选型

Step 3 — 后端核心服务开发

Step 4 — REST API 与前后端联调

Step 5 — 桌面应用打包与部署

Step 6 — 性能调优与问题修复

第三部分：核心代码解析

3.1 数据流架构

3.2 关键算法

3.3 API 接口文档

第四部分：安装与运行手册

4.1 环境要求

4.2 从 GitHub 获取源码

4.3 后端安装与启动

4.4 前端安装与启动

4.5 桌面应用打包

4.6 常见问题

第五部分：项目结构

第一部分：项目概述

1.1 项目定位

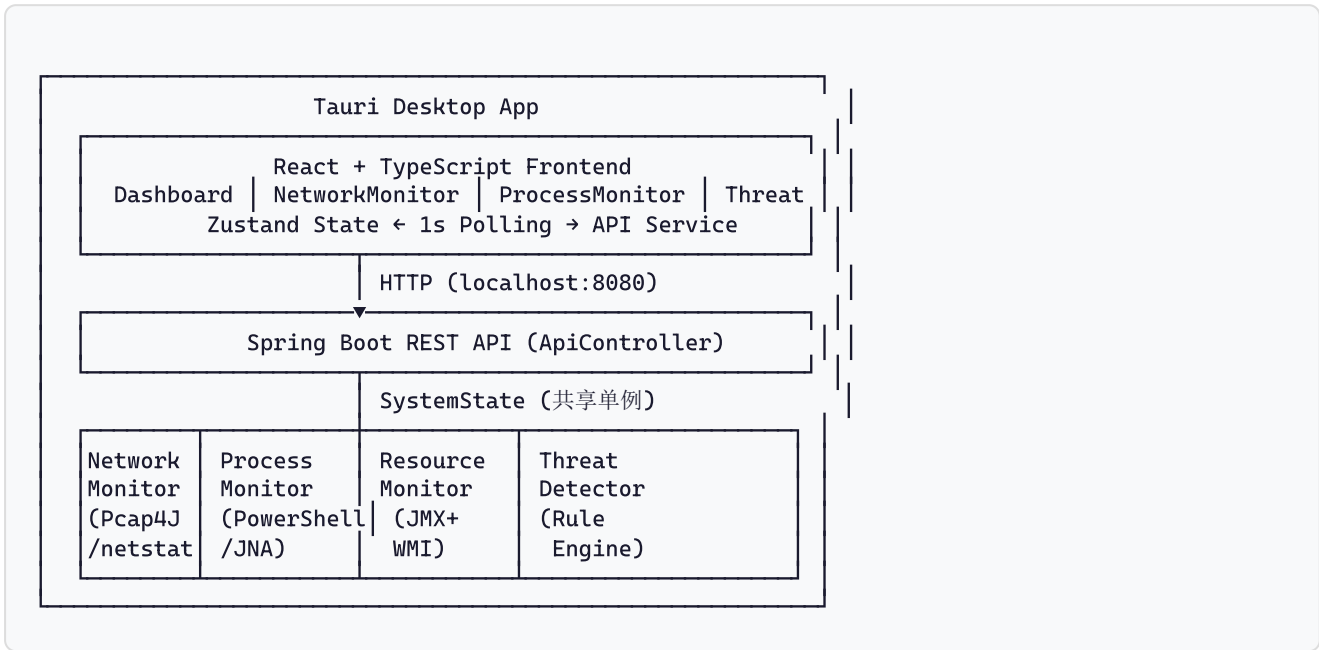
DKail 是一款面向安全学习场景的轻量级主机安全监控系统。项目采用 Java (Spring Boot) 后端 + React (Tauri) 前端架构，提供实时网络流量监控、进程监控、系统资源监控及威胁检测功能，最终打包为原生桌面应用。

本项目全程采用 AI Agent 协同开发模式，本文档将完整呈现人机协作的开发过程，供学习者参考。

1.2 功能模块

模块	功能	后端实现	前端页面
网络监控	实时流量捕获、协议解析、连接列表	NetworkMonitor	NetworkMonitor.tsx
进程监控	进程列表、CPU/内存占用排行	ProcessMonitor	ProcessMonitor.tsx
资源监控	CPU/内存/磁盘使用率、网络速率	SystemResourceMonitor	Dashboard.tsx
威胁检测	可疑进程识别、异常告警	ThreatDetector	ThreatDetection.tsx

1.3 技术架构



第二部分：AI Agent 协同开发过程

本项目全程采用 AI Agent 协同开发模式（基于 Trae IDE），遵循“需求→设计→实现→验证”的迭代流程。以下是完整的开发步骤，每一步均包含开发者输入的 Prompt 和 Agent 的执行逻辑。

开发者 → Agent → 产出物 → 开发者 Review → 下一步（或反馈修改）
Step 1: 需求分析 → PRD + 伪代码 → Review
Step 2: 架构设计 → 伪代码细化 → Review
Step 3: 后端开发 → Java 服务层代码 → 编译验证
Step 4: 前端开发 → React + TypeScript 页面 → 页面验证
Step 5: API 联调 → 前后端数据打通 → 运行验证
Step 6: 桌面打包 → Tauri EXE → 功能验证
Step 7: 性能调优 → 修复与优化 → 数据验证

1

需求分析与 PRD 编写

开发者 → Agent

目标

明确项目目标、功能范围、技术选型，输出产品需求文档（PRD）。

开发者 Prompt

[开发者] 我要开发一款主机安全监控系统 DKail，目标平台为 Windows，需要实现以下功能模块：

- 网络流量监控：实时捕获网络数据包，解析 TCP/UDP 协议，展示活跃连接列表和流量统计
- 进程监控：获取系统进程列表，展示 CPU/内存占用排行，识别高资源消耗进程
- 系统资源监控：实时采集 CPU、内存、磁盘使用率及网络速率
- 威胁检测：基于规则引擎检测可疑进程和异常行为，生成告警

技术栈选型：

- 后端：Java 17 + Spring Boot 3.2，使用 Pcap4J 进行网络抓包，JNA 调用系统 API，WMI/PowerShell 获取进程和资源数据
- 前端：React 18 + TypeScript + Vite，使用 Zustand 状态管理，Tailwind CSS 样式，Tauri 2 打包为桌面应用
- 构建工具：Maven（后端）、npm（前端）

请帮我生成完整的 PRD 文档和项目伪代码，覆盖所有模块的类结构、方法签名、数据流和 API 设计。

Agent 执行逻辑

- 分析需求关键词：安全监控、网络/进程/资源/威胁、Java后端、Tauri桌面
- 生成 PRD 文档，包含：项目背景、功能模块拆分、技术栈选型、数据模型设计、API 设计草案

3. 输出伪代码文件 DKail项目开发伪代码.md，作为后续开发的蓝图

开发者 Review

Agent 输出 PRD 和伪代码后，开发者需要审核以下关键点：

- **功能完整性**：4 大模块是否覆盖了所有需求？是否有遗漏的监控维度？
- **技术选型合理性**：Pcap4J 是否适合当前场景？JNA 调用是否有平台限制？
- **数据模型设计**：SystemState 是否能承载所有监控数据？字段类型是否合理？
- **API 设计**：端点是否满足前端数据需求？字段命名是否一致？
- **可行性评估**：Windows 环境下 PowerShell/WMI 调用是否可靠？Tauri 打包是否有兼容性风险？

Review 通过后，进入 Step 2 正式编写代码；如有问题，向 Agent 反馈修改意见并重新生成。

产出物

- 项目功能模块划分（4大模块）
- 技术栈确认：Java 17 + Spring Boot 3.2 + Pcap4J + JNA + React 18 + Tauri 2
- 数据模型草案：SystemState、Resources、Alert、ProcessInfo、ConnectionInfo 等
- API 端点设计草案

2

架构设计与伪代码编写

开发者 → Agent

目标

基于 PRD 设计系统架构，编写完整的伪代码，确定每个模块的实现逻辑和接口协议。

开发者 Prompt

【开发者】 PRD 已确认，现在进入架构设计阶段。请基于 PRD 编写完整的伪代码，要求：

后端架构：

- 采用 Spring Boot 分层架构：Model → Service → Controller
- SystemState 作为共享状态单例，所有 Service 通过它交换数据
- 4 个监控服务通过 ScheduledExecutorService 并发运行
- NetworkMonitor: Pcap4J 抓包 + netstat 降级方案
- ProcessMonitor: PowerShell Get-Process 两次采样差值法计算 CPU%
- SystemResourceMonitor: JMX 反射获取 CPU/内存 + WMI 获取网络速率
- ThreatDetector: 规则引擎匹配可疑进程名 + 阈值检测

前端架构：

- React 18 + TypeScript, 组件化设计
- Zustand 全局状态管理, 1秒轮询后端 API
- 4 个页面: Dashboard、NetworkMonitor、ProcessMonitor、ThreatDetection
- Tailwind CSS 暗色主题, 适配安全监控场景
- Tauri 2 打包为 Windows 桌面应用

API 设计:

- 6 个 REST 端点, snake_case 字段命名
- Vite 开发代理转发到 Spring Boot 8080 端口

请输出完整伪代码文件 DKail项目开发伪代码.md

Agent 执行逻辑

1. 基于 PRD 中的功能模块, 设计分层架构 (Model → Service → Controller)
2. 按 Spring Boot 项目规范规划 Maven 项目结构
3. 设计 13 个 Model 类、4 个 Service 类、1 个 Controller、1 个 Application 入口
4. 设计前端 React + TypeScript 组件结构 (6 个组件、4 个页面、Store、API 服务)
5. 设计 Tauri 配置、Vite 配置、Tailwind 配置
6. 输出完整伪代码文件

产出物

- 系统架构设计文档
- 完整伪代码文件 DKail项目开发伪代码.md
- Maven 项目结构规划
- 前端组件结构规划

3

后端核心服务开发

开发者 → Agent

目标

基于伪代码, 实现四大监控服务的核心数据采集逻辑, 确保在 Windows 环境下可运行。

开发者 Prompt

[开发者] 伪代码已审核通过, 现在实现后端所有 Java 代码。项目目录: Dkail-java/

要求:

- Maven 项目, Spring Boot 3.2.0, Java 17, Lombok
- 依赖: spring-boot-starter-web、pcap4j-core、pcap4j-packetfactory-static、jna

- 编码 UTF-8，避免 Windows GBK 乱码
- 13 个 Model 类放在 `com.dkail.model` 包下
- 4 个 Service 类放在 `com.dkail.service` 包下，实现 `Runnable` 接口
- 1 个 ApiController 放在 `com.dkail.controller` 包下，6 个 GET 端点
- DkailApplication 主入口通过 `CommandLineRunner` 启动 4 个监控线程
- SystemState 使用 `synchronized` 集合保证线程安全
- NetworkMonitor 需要处理 Pcap4J 不可用时的降级逻辑
- ProcessMonitor 使用两次采样差值法计算 CPU%，需除以 CPU 核心数
- SystemResourceMonitor 通过 WMI 两次采样计算网络速率（bytes/s）
- ThreatDetector 需要告警去重机制

请生成完整的 Maven 项目结构和所有 Java 源文件。

关键实现细节

3.1 NetworkMonitor — 网络监控

- **主模式**：Pcap4J 抓包，实时解析 TCP/UDP 数据包
- **降级模式**：无 Npcap 时自动切换为 `netstat -an` 解析
- 通过 SystemState 共享连接列表和数据包统计

```
// 自动检测 Npcap 可用性
try {
    PcapHandle handle = Pcap.openLive(deviceName, snapLen, mode, timeout);
    // Pcap4J 模式
} catch (Exception e) {
    log.warn("Pcap4J unavailable, falling back to netstat mode");
    // netstat 降级模式
}
```

3.2 ProcessMonitor — 进程监控

- 通过 PowerShell `Get-Process` 获取进程列表
- **CPU 使用率**：两次采样差值法，除以 CPU 核心数
- 按 CPU 降序排列，取 Top 100 进程

$$\text{CPU\%} = (\Delta \text{累计CPU时间} / \Delta \text{采样间隔}) / \text{CPU核心数} \times 100$$

3.3 SystemResourceMonitor — 资源监控

- CPU/内存：反射调用 `com.sun.management.OperatingSystemMXBean`
- 磁盘：`File.getTotalSpace()` / `File.getFreeSpace()`
- **网络速率**：WMI `Win32_PerfRawData_Tcpip_NetworkInterface` 两次采样差值

$\text{Rate(B/s)} = (\Delta \text{累计字节数} / \Delta \text{采样间隔})$

3.4 ThreatDetector — 威胁检测

- 规则引擎：可疑进程名匹配（mimikatz、nc、cobaltstrike 等）
- 阈值检测：CPU > 80%、内存 > 500MB 触发告警
- 告警去重：alertedProcesses 集合防止重复

3.5 DkailApplication — 主入口

```
@Bean
public CommandLineRunner startMonitors(SystemState state) {
    return args -> {
        ScheduledExecutorService pool = Executors.newScheduledThreadPool(4);
        pool.submit(new NetworkMonitor(state));
        pool.submit(new ProcessMonitor(state));
        pool.submit(new SystemResourceMonitor(state));
        pool.submit(new ThreatDetector(state));
    };
}
```

4

前端界面开发

开发者 → Agent

目标

基于伪代码中的前端架构设计，实现 React + TypeScript 前端界面，包含 4 个监控页面和通用组件。

开发者 Prompt

[开发者] 后端代码已完成，现在开发前端界面。项目目录：Dkail-java/dkail-ui/

要求：

- Vite + React 18 + TypeScript 项目
- Zustand 全局状态管理，1秒轮询后端 API（<http://127.0.0.1:8080>）
- Tailwind CSS 暗色主题（#0f172a 主背景），适配安全监控场景
- 4 个页面组件：
 - Dashboard: 系统概览，4个统计卡片 + 实时流量图表 + CPU/内存/磁盘仪表盘 + 告警列表
 - NetworkMonitor: 实时流量图表 + 活跃连接表格 + 数据包统计
 - ProcessMonitor: 进程列表表格（PID/进程名/CPU%/内存）+ 搜索过滤 + 排序
 - ThreatDetection: 威胁等级统计 + 告警列表 + 威胁分布图
- 6 个通用组件：Header、ConnectionStatus、ResourceGauge、StatCard、AlertItem、T

rafficChart

- TypeScript 类型定义与后端 API 响应字段一一对应 (snake_case)
- Vite 代理配置: /status、/resources、/processes、/network、/alerts、/health 转发到 8080 端口
- 左侧导航栏 + 主内容区布局

请生成完整的前端项目代码。

Agent 执行逻辑

1. 初始化 Vite + React + TypeScript 项目
2. 配置 Tailwind CSS、PostCSS、TypeScript
3. 实现 TypeScript 类型定义 (types/index.ts)
4. 实现 Zustand Store 和 API 服务层
5. 实现 6 个通用组件
6. 实现 4 个页面组件
7. 配置 Vite 代理

产出物

- 完整前端项目源码 (可 npm run dev 运行)
- 4 个监控页面 + 6 个通用组件
- Zustand Store + API 服务层 + TypeScript 类型
- Vite/Tailwind/PostCSS/TypeScript 配置文件

5

REST API 与前后端联调

开发者 → Agent

目标

实现 REST API，确保后端数据格式与前端 TypeScript 类型定义完全匹配，完成前后端数据联调。

开发者 Prompt

[开发者] 前后端代码都已写好，但 API 对接有问题。请检查前端 api.ts 和 types/index.ts 中定义的字段名和类型，调整后端 ApiController 的返回格式，确保：

1. 所有 JSON 字段名使用 snake_case (如 cpu_usage、memory_usage、network_in)
2. /status 返回 network_monitor_active、process_monitor_active、threat_detection_active、alert_count、uptime

3. `/resources` 返回 `cpu_usage`、`memory_usage`、`disk_usage`、`network_in` (bytes/s 速率)、`network_out` (bytes/s 速率)
4. `/processes` 返回 `processes` 数组，每个元素含 `pid`、`name`、`path`、`cpu_usage`、`memory_usage`
5. `/network` 返回 `connections` 数组 + `packet_count` + `byte_count`
6. `/alerts` 返回 `alerts` 数组 + `count`
7. `/health` 返回 `status` + `timestamp`

同时确认 Vite 代理配置正确转发到 Spring Boot 8080 端口。

Agent 执行逻辑

1. 读取前端 `api.ts` 和 `types/index.ts`，提取所有 API 调用和字段名
2. 调整 `ApiController`，手动构建 `snake_case` JSON 键名匹配前端
3. 配置 Vite 代理，将 API 请求转发到 Spring Boot 后端
4. 启动后端 + 前端，逐个验证 API 响应

API 端点与前端对应关系

API 端点	前端调用位置	关键字段
GET <code>/status</code>	App.tsx → <code>fetchStatus()</code>	<code>network_monitor_active</code> , <code>alert_count</code> , <code>uptime</code>
GET <code>/resources</code>	App.tsx → <code>fetchResources()</code>	<code>cpu_usage</code> , <code>memory_usage</code> , <code>disk_usage</code> , <code>network_in</code> , <code>network_out</code>
GET <code>/processes</code>	App.tsx → <code>fetchProcesses()</code>	<code>pid</code> , <code>name</code> , <code>cpu_usage</code> , <code>memory_usage</code>
GET <code>/network</code>	App.tsx → <code>fetchNetwork()</code>	<code>connections</code> , <code>packet_count</code> , <code>byte_count</code>
GET <code>/alerts</code>	App.tsx → <code>fetchAlerts()</code>	<code>alerts</code> , <code>count</code>
GET <code>/health</code>	App.tsx → 健康检查	<code>status</code> , <code>timestamp</code>

⚠ 前端使用 `snake_case` 字段名 (如 `cpu_usage`)，Java 默认 `camelCase` (如 `cpuUsage`)，因此 Controller 中需手动构建 Map 确保 JSON 键名匹配。

目标

将 React 前端通过 Tauri 2 打包为 Windows 桌面 EXE 安装程序。

开发者 Prompt

[开发者] 前端开发完成，现在需要打包为桌面应用。请配置 Tauri 2 并构建 Windows 安装程序：

要求：

- 在 `dkail-ui/` 下配置 `src-tauri/` 目录
- `tauri.conf.json` 中 `devUrl` 指向 `http://127.0.0.1:5173`
- 应用名称：DKail Security Monitor
- 窗口大小：1280x800，可调整，暗色标题栏
- 生成图标集：`npx tauri icon icon.ico`
- 构建命令：`npm run tauri build`
- 输出：独立 EXE + NSIS 安装程序 + MSI 安装包

运行时需先启动 Java 后端（`java -jar dkail-1.0.0.jar`），桌面应用通过 HTTP 调用后端 API。

Agent 执行逻辑

1. 配置 `tauri.conf.json`：devUrl 指向 Vite 5173 端口
2. 生成 Tauri 图标集：`npx tauri icon icon.ico`
3. 执行 `npm run tauri build` 编译 Rust + 打包

产出物

- `dkail-ui.exe` — 独立可执行文件
- `DKail Security Monitor_1.0.0_x64-setup.exe` — NSIS 安装程序
- `DKail Security Monitor_1.0.0_x64_en-US.msi` — MSI 安装包

7

性能调优与问题修复

开发者反馈 → Agent 修复

目标

系统运行后，根据实际监控数据验证结果，修复开发过程中的逻辑问题。

开发者 Prompt (逐轮反馈)

【开发者 - 第1轮】 进程监控页面 CPU 总使用率显示 1543.5%，这明显不对。PowerShell Get-Process 返回的 CPU 字段是累计时间而非使用率百分比，请用两次采样差值法计算，并除以 CPU 核心数，使各进程 CPU% 加和约等于系统总 CPU 使用率。

【开发者 - 第2轮】 实时网络流量图只增不减，network_in/network_out 应该是实时速率（bytes/s），不是累计总字节数。请通过 WMI Win32_PerfRawData_Tcpip_NetworkInterface 两次采样差值计算速率。

【开发者 - 第3轮】 没有 Npcap 时后端启动报错崩溃，NetworkMonitor 需要自动检测 Pcap4J 可用性，不可用时降级为 netstat 模式。

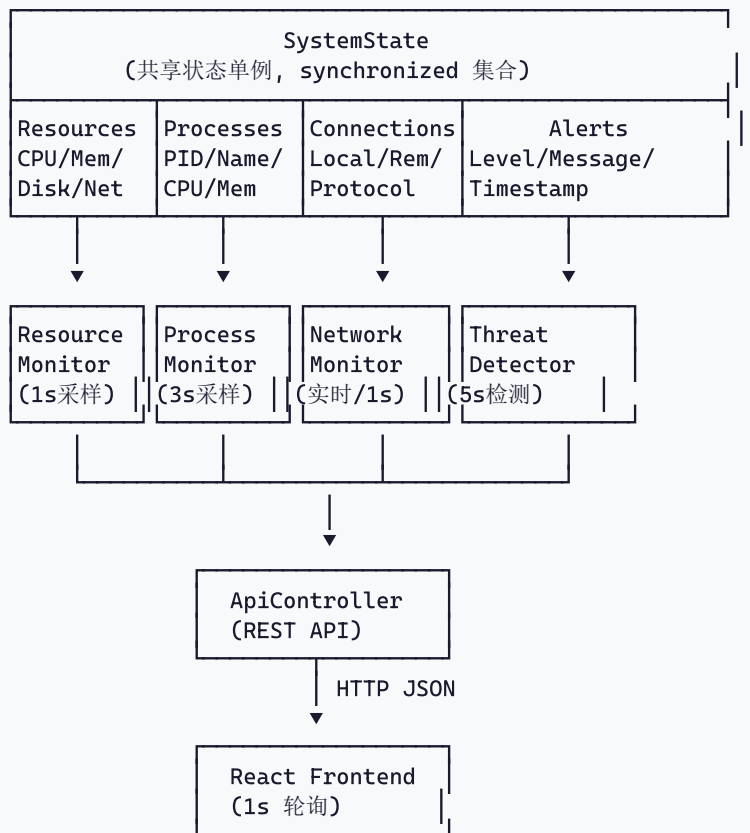
问题修复记录

问题	根因	修复方案
CPU 占比 1543.5%	PowerShell Get-Process 的 CPU 字段是累计时间（秒），非使用率	两次采样差值法 + 除以核心数
各进程 CPU 加和远大于系统总 CPU	计算的是单核占比，未除以核心数	$CPU\% = \Delta CPU时间 / \Delta 间隔 / 核心数 \times 100$
网络流量图只增不减	network_in/out 是累加值，非速率	WMI 两次采样差值计算 bytes/s 速率
Pcap4J 无 Npcap 时崩溃	缺少 try-catch 降级	自动检测 + netstat 回退模式
Spring Boot 编码 GBK 乱码	Windows 默认 GBK	pom.xml 添加 <encoding>UTF-8</encoding>
告警重复生成	无去重机制	添加 alertedProcesses 集合

💡 Agent 辅助开发中，"运行→发现问题→反馈→修复"是核心迭代模式。开发者需要理解 Agent 输出的逻辑，并在运行验证时发现和纠正问题。

第三部分：核心代码解析

3.1 数据流架构



3.2 关键算法

进程 CPU 使用率 (ProcessMonitor.java)

PowerShell `Get-Process` 返回的 CPU 字段是累计 CPU 时间（秒），不是使用率百分比。通过两次采样差值法计算：

$$\text{CPU\%} = (\text{本次累计CPU时间} - \text{上次累计CPU时间}) / (\text{本次时间} - \text{上次时间}) / \text{CPU核心数} \times 100$$

除以核心数后，各进程 CPU% 加和约等于任务管理器中的总 CPU 使用率。

网络速率 (SystemResourceMonitor.java)

WMI `Win32_PerfRawData_Tcpip_NetworkInterface` 的 `BytesReceivedPersec/BytesSentPersec` 是累计原始计数器值，两次采样差值除以时间得到实时速

率:

$$\text{Rate(B/s)} = (\text{本次累计字节数} - \text{上次累计字节数}) / (\text{本次时间} - \text{上次时间})$$

3.3 API 接口文档

GET /status

系统运行状态。

```
{
  "network_monitor_active": true,
  "process_monitor_active": true,
  "threat_detection_active": true,
  "alert_count": 3,
  "uptime": 3600
}
```

GET /resources

系统资源使用率。network_in/network_out 为实时速率 (bytes/s) 。

```
{
  "cpu_usage": 23.5,
  "memory_usage": 78.0,
  "disk_usage": 80.1,
  "network_in": 505202,
  "network_out": 25376
}
```

GET /processes

进程列表, 按 CPU 降序, 最多 100 条。

```
{
  "processes": [
    {"pid": 1234, "name": "java.exe", "path": "", "cpu_usage": 3.5, "memory_usage": 256.8},
  ],
  "count": 100
}
```

GET /network

网络连接与统计。

```
{
  "connections": [
    {"local_addr": "192.168.1.5:5173", "remote_addr": "127.0.0.1:8080", "protocol":
"TCP", "state": "ESTABLISHED"}
  ],
  "packet_count": 27752,
  "byte_count": 23200000
}
```

GET /alerts

告警列表。

```
{
  "alerts": [
    {"id": 1, "timestamp": "2026-06-12T19:30:00", "alert_type": "SuspiciousActivity",
      "severity": "High", "source": "system",
      "description": "Detected suspicious process: nc.exe", "details": {}}
  ],
  "count": 1
}
```

GET /health

```
{"status": "ok", "timestamp": 1718192400000}
```

第四部分：安装与运行手册

4.1 环境要求与安装

JDK 17

Java 运行与编译环境。

```
# 下载安装
# 访问 https://adoptium.net/ 下载 JDK 17 Windows x64 MSI 安装包
# 安装后验证
java -version
# 预期输出: openjdk version "17.x.x"

# 手动配置环境变量（安装程序通常自动配置）
# JAVA_HOME = C:\Program Files\Eclipse Adoptium\jdk-17.x.x
# Path 中添加 %JAVA_HOME%\bin
```

Maven 3.9+

Java 后端项目构建工具。

```
# 下载安装
# 访问 https://maven.apache.org/download.cgi 下载 apache-maven-3.9.x-bin.zip
# 解压到 C:\Program Files\Apache\maven

# 配置环境变量
# MAVEN_HOME = C:\Program Files\Apache\maven
# Path 中添加 %MAVEN_HOME%\bin

# 验证
mvn -version
# 预期输出: Apache Maven 3.9.x
```

Node.js 18+

前端构建与依赖管理环境（npm 随 Node.js 一起安装）。

```
# 下载安装
# 访问 https://nodejs.org/ 下载 LTS 版本 Windows Installer
# 安装时勾选 "Add to PATH"

# 验证
node -version
# 预期输出: v18.x.x 或更高
```



```
npm -version
# 预期输出: 9.x.x 或更高
```

Rust 1.70+

Tauri 桌面应用编译工具链。

```
# 下载安装
# 访问 https://rustup.rs/ 下载 rustup-init.exe
# 运行安装程序, 选择默认安装 (1)

# 配置环境变量 (安装程序通常自动配置)
# Path 中添加 %USERPROFILE%\cargo\bin

# 验证
rustc --version
# 预期输出: rustc 1.7x.x
cargo --version
# 预期输出: cargo 1.7x.x
```

Git

源码版本管理工具。

```
# 下载安装
# 访问 https://git-scm.com/download/win 下载 Git for Windows Installer
# 安装时使用默认选项

# 验证
git --version
# 预期输出: git version 2.x.x
```

Npcap (可选)

网络数据包捕获驱动。未安装时系统自动降级为 netstat 模式, 不影响基本功能。

```
# 下载安装
# 访问 https://npcap.com/ 下载 Npcap 安装程序
# 安装时勾选 "Install Npcap in WinPcap API-compatible Mode"
```

4.2 快速启动 (使用预构建 EXE)

项目提供了预构建的桌面安装程序, 无需编译即可使用。

步骤一: 下载安装包

从 GitHub Releases 页面下载最新版本安装程序:

```
# 访问 https://github.com/chenlong548/DKail/releases
# 下载 DKail Security Monitor_1.0.0_x64-setup.exe (NSIS 安装程序)
# 或 DKail Security Monitor_1.0.0_x64_en-US.msi (MSI 安装包)
```

步骤二：安装并启动后端

桌面应用依赖 Java 后端，需先启动后端服务。从 Releases 下载 dkail-1.0.0.jar：

```
# 确保 JDK 17 已安装
java -jar dkail-1.0.0.jar
```

启动成功后，后端监听 `http://127.0.0.1:8080`。

步骤三：启动桌面应用

双击桌面快捷方式或在开始菜单中启动 DKail Security Monitor，界面将自动连接后端 API。

⚠ 桌面应用运行时必须保持 Java 后端服务运行。如后端未启动，界面将显示连接失败。

4.3 从源码构建

步骤一：获取源码

```
git clone https://github.com/chenlong548/DKail.git
cd DKail/Dkail-java
```

步骤二：构建后端

```
mvn clean package -DskipTests
```

构建成功后生成 `target/dkail-1.0.0.jar`。

步骤三：启动后端服务

```
java -jar target/dkail-1.0.0.jar
```

可通过以下命令验证后端是否正常运行：

```
curl http://127.0.0.1:8080/health
# 预期返回: {"status":"ok","timestamp": ... }
```

步骤四：安装前端依赖

```
cd dkail-ui
npm install
```

步骤五：启动前端开发服务器

```
npm run dev
```

浏览器访问 <http://127.0.0.1:5173> 即可看到监控界面。

💡 开发模式下，Vite 会自动将 API 请求代理到后端 8080 端口，无需额外配置。

步骤六：构建桌面应用

```
# 在 dkail-ui 目录下
npm run tauri build
```

构建完成后，安装包位于：

文件类型	路径
独立 EXE	src-tauri/target/release/dkail-ui.exe
NSIS 安装程序	src-tauri/target/release/bundle/nsis/DKail Security Monitor_1.0.0_x64-setup.exe
MSI 安装包	src-tauri/target/release/bundle/msi/DKail Security Monitor_1.0.0_x64_en-US.msi

4.4 常见问题

Q: 启动后端报 Pcap4J 相关错误？

A: 系统会自动检测 Npcap 是否安装。未安装时自动降级为 netstat 模式，不影响基本功能。如需完整抓包能力，请安装 [Npcap](#)。

Q: 进程 CPU 使用率首次显示为 0？

A: 首次采样无法计算差值，第二次采样（约 3 秒后）开始显示真实 CPU 使用率。

Q: 网络速率首次显示为 0？

A: 同上，首次采样无差值可计算，约 1 秒后开始显示实时速率。

Q: Tauri 构建报 cargo not found？

A: 需将 Rust 的 cargo 路径加入系统 PATH:

```
$env:Path = "$env:USERPROFILE\.cargo\bin;$env:Path"
```

Q: Tauri 构建报图标缺失?

A: 运行 `npx tauri icon <图标路径>` 生成完整的图标文件集。

Q: 前端页面空白或无法连接后端?

A: 确认后端已启动 (访问 `http://127.0.0.1:8080/health` 返回 OK) , 开发模式下 Vite 代理会自动转发 API 请求。

第五部分：项目结构

```
Dkail-java/
├── pom.xml # Maven 配置
├── src/
│   ├── main/
│   │   ├── java/com/dkail/
│   │   │   ├── DkailApplication.java # 主入口（线程池启动4个服务）
│   │   │   └── controller/
│   │   │       └── ApiController.java # REST API（6个端点）
│   │   │   └── model/ # 数据模型（13个类）
│   │   │       ├── SystemState.java # 系统全局状态
│   │   │       ├── Resources.java # 资源使用率
│   │   │       ├── Alert.java # 告警信息
│   │   │       ├── AlertLevel.java # 告警级别枚举
│   │   │       ├── ConnectionInfo.java # 网络连接信息
│   │   │       ├── ParsedPacket.java # 解析后的数据包
│   │   │       ├── ProcessSummary.java # 进程摘要
│   │   │       ├── ProcessInfo.java # 进程详细信息
│   │   │       ├── SystemStatus.java # 系统运行状态
│   │   │       ├── ThreatIndicator.java # 威胁指标
│   │   │       ├── ThreatCategory.java # 威胁分类枚举
│   │   │       ├── TrafficDirection.java # 流量方向枚举
│   │   │       └── Severity.java # 严重程度枚举
│   │   └── service/ # 核心服务（4个）
│   │       ├── NetworkMonitor.java # 网络监控
│   │       ├── ProcessMonitor.java # 进程监控
│   │       ├── SystemResourceMonitor.java # 资源监控
│   │       └── ThreatDetector.java # 威胁检测
│   └── resources/
│       ├── application.yml # Spring Boot 配置
│       └── banner.txt # 启动横幅
├── test/ # 测试代码
└── dkail-ui/ # 前端项目
    ├── package.json
    ├── vite.config.ts # Vite + API 代理配置
    ├── src/
    │   ├── App.tsx # 主应用（数据轮询逻辑）
    │   ├── services/api.ts # API 服务层
    │   ├── store/index.ts # Zustand 状态管理
    │   ├── types/index.ts # TypeScript 类型定义
    │   └── pages/ # 页面组件
    │       ├── Dashboard.tsx # 仪表盘
    │       ├── NetworkMonitor.tsx # 网络监控
    │       ├── ProcessMonitor.tsx # 进程监控
    │       ├── ThreatDetection.tsx # 威胁检测
    │       └── Settings.tsx # 系统设置
    ├── components/ # 通用组件
    ├── layouts/MainLayout.tsx # 主布局
    └── src-tauri/ # Tauri 桌面应用配置
```

```
├── tauri.conf.json
├── Cargo.toml
├── src/
├── DKail-Project-Presentation.html # 项目演示 PPT
└── 开发与安装手册.html # 本文档
```

DKail Security Monitor — 开发与安装手册

Developer: 陈美龙 | 本项目仅供学习与交流使用，所有版权归原作者所有